Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of deep learning, providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the reference work *Deep Learning* [**?**]. Its structure is as follows:

Its structure is as follows:

Section 1.1: An overview of the milestones that led to the emergence of reinforcement learning as a unified field.

Section 1.2: A formalization of the objective underlying reinforcement learning algorithms.

Section 1.3: An explanation of how policies are progressively refined through interaction with the environment to approach the optimal policy.

1.1 Historical Developments

Reinforcement learning is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The origins of reinforcement learning can be traced back to 20th-century animal psychology studies [**?**]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior through repeated interaction with the environment [**?**]. The term "reinforcement" in the context of learning comes from this school of thought, where it refers precisely to the process of strengthening a behavior through its consequences.

Compiled on 30/6/2026 at 13:55

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a "pleasure–pain system" [**?**]. Although these studies did not involve formal mathematical models, they introduced the central idea of reinforcement learning, trial-and-error learning guided by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second foundation of reinforcement learning comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making [**?**]. MDPs describe a decision-maker interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDPs is dynamic programming [**?**]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic programming requires a full and exact description of the environment, which is only feasible for small problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference learning methods were introduced [**?**]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [**?**, **?**].

Shortly before the turn of the millennium, the publication of the foundational reference work [**?**] contributed to the formalization of reinforcement learning as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

1.1.4 Consolidation of reinforcement learning (2000–2013)

A limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the

full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDPs, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced. These allowed value functions and policies to be represented using parametric models such as linear approximators or perceptron-like architectures [**?**, **?**]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space. The introduction of Deep Q-Networks (DQN), which demonstrated that deep Neural Network (NN) could be used to approximate value functions at scale [**?**].

1.1.5 Deep reinforcement learning (2013–2026)

The introduction of deep learning into reinforcement learning marked a major turning point in the field. A series of algorithms significantly improved scalability. Deep Deterministic Policy Gradient (DDPG) extended actor–critic methods to continuous action spaces [**?**]. Proximal Policy Optimization (PPO) later became a standard algorithm in reinforcement learning due to its simplicity, stability, and ability to perform in both continuous and discrete action spaces [**?**].

These advances established a practical foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, reinforcement learning, and tree search could reach superhuman performance in the game of Go [**?**]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [**?**]. Similarly, in Dota 2, large-scale multi-agent reinforcement learning systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [**?**].

More recently, reinforcement learning has expanded beyond its traditional application domains. With the emergence of increasingly agentic views of large language models, it has become possible to train these models to perform specific tasks using reinforcement learning-based techniques. In particular, reinforcement learning from human feedback has become a method for aligning large language models with human preferences. It does so by formalizing the problem as an MDP and using reinforcement learning to solve it [**?**]. Similarly, DeepSeekMath shows how reinforcement learning can enhance the mathematical reasoning abilities of large language models [**?**]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a large language model with reinforcement learning on mathematical problem-solving can also enhance its programming capabilities.

Originally inspired by biological learning processes, reinforcement learning is now closer than ever to this initial intuition. Agents learn through interaction, adapt via trial and error, and progressively improve their behavior based on experience. This behavior is governed by deep neural architectures, which are the

closest computational analog to organic brains. More than ever, reinforcement learning stands as one of the core paradigms for achieving artificial general intelligence [**?**].

1.2 Objective

Now that the main developments shaping reinforcement learning have been outlined, the formal definition of its objective can be introduced. In essence, reinforcement learning aims to build a system that makes sequential decisions in order to maximize a cumulative reward signal over the long term. The purpose of this section is to translate this intuitive goal into a rigorous mathematical framework. To this end, the presentation proceeds step by step:

1. Problems are modeled using Markov Decision Processes in Subsection 1.2.1.
2. The notion of policy, which formalizes the agent's behavior, is introduced in Subsection 1.2.2.
3. The interaction between the agent and the environment is characterized through trajectories in Subsection 1.2.3.
4. Policies are evaluated using value functions in Subsection 1.2.4.
5. The optimal policy is defined in Subsection 1.2.5.

1.2.1 Markov Decision Processes

The RL framework is grounded in the agent paradigm: the decision-making entity is called the agent. The agent perceives its environment, processes the information it receives, and acts upon the environment to influence future outcomes. To specify the task and the agent's capacity to perceive and act, the problem is modeled as a Markov Decision Process (MDP).

Before providing a more detailed definition, it is useful to clarify the scope of the MDP problems considered in this manuscript. The goal of this subsection is to introduce the notations and formalisms as they are used in modern RL practice. Other formulations of MDPs exist that are better suited, for instance, to proving convergence guarantees. Our focus here is to represent reinforcement learning as it is applied, even if this means sacrificing some mathematical proof capability. We therefore restrict the analysis to the following configuration:

- The single-agent learning setting is considered. If any other agents are present in the environment, they are treated as part of the environment dynamics.
- Finite-horizon MDPs are considered. Each interaction sequence between the MDP and the policy has a defined beginning and a defined end. Between these two points, the number of interaction steps may vary depending on the situation, but it always remains finite.
- The partially observable framework is adopted, where the agent does not have direct access to the true state of the system, but instead relies on limited observations that provide only partial information about the environment.

The resulting framework can be formally described as a single-agent finite-horizon partially observable markov decision process. For simplicity, we will refer to this setting as an "MDP" throughout this manuscript. A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- $\mathcal{S}$: The set of states, representing all possible configurations of the environment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, two special subsets are distinguished: initial states, from which an interaction can begin, and terminal states, in which the interaction ends. The set of initial states is denoted by $\mathcal{S}^i$, with an initial state written as $s^i \in \mathcal{S}^i$. The set of terminal states is denoted by $\mathcal{S}^t$, with a terminal state written as $s^t \in \mathcal{S}^t$.
- $\mathcal{O}$: The set of observations that the agent can perceive from the environment. A observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike the true state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$ provides partial information about the current state.
- $\mathcal{A}$: The set of actions that the agent may execute. A action belonging to this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP, thereby shaping its future course. They represent the agent's capacity to act upon its environment.
- $\mathcal{R}$: The set of rewards that the agent can receive from the environment. A reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$. The agent's goal is to maximize the cumulative sum of these rewards over time.
- p: The transition function, which defines the dynamics of the environment. Given a state s and an action a, this function defines a probability distribution over the joint outcome consisting of the next state s', the observation o, and the reward r. This function is denoted by p, and is defined as: $p : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$. The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple (s', o, r) under the distribution $p(\cdot \mid s, a)$.

1.2.2 Policy

The agent's decision-making capability is modeled by a policy function. The policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions $o \in \mathcal{A}$. A policy function is denoted by π, and is defined as: $\pi : \mathcal{O} \to \mathcal{P}(\mathcal{A})$. The notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot \mid o)$.

1.2.3 Trajectory

Now that both the MDP and the policy π have been defined, the interaction between them can be characterized. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s, provides an observation o to the policy π. Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot \mid o)$, from which an action is sampled: $a \sim \pi(\cdot \mid o)$.

2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot \mid s, a)$. It moves to a new state s', and returns a new observation o' and a reward r.

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. The distribution over trajectories is denoted by τ, and a particular trajectory is denoted by t. The distribution τ is determined by the initial state s, the initial observation o, the environment dynamics p, and the policy π, and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s, o, p, and π.

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2. Specifically, the probability of a trajectory t factorizes as:

$$\tau(t \mid s_1, o_1, p, \pi) = \pi(a_1 \mid o_1)\, p(s_2, o_2, r_1 \mid s_1, a_1)\, \tau(t_{2:} \mid s_2, o_2, p, \pi),$$

where $t_{2:} = (s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau_t(s^i, o, p, \pi)$.

1.2.4 Value Functions

To evaluate the ability of a policy π to collect rewards, the notion of return is introduced. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{T-1} r_T = \sum_{i=1}^{T} \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, the discount factor would ideally be set to $\gamma = 1$. In practice, however, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain. Throughout this

manuscript, unless explicitly stated otherwise, the discount factor is assumed to be $\gamma = 1$ to simplify notation.

The return also admits a recursive form, which is central to the dynamic programming and reinforcement learning algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma\, g(t_{2:}),$$

From the notion of return, functions that estimate the expected return of a policy from a given starting point can be defined. These functions are called value functions. The first one considered in detail is the state-value function V. It tells us how desirable a particular state is under a given policy π, given an observation o and environment dynamics p. Strictly speaking, this function should be written as $V(s, o, p, \pi)$ to reflect all its dependencies. However, since the environment dynamics p are fixed throughout learning, they are omitted from the notation. To emphasize that the value is tied to a specific policy, π is placed as a subscript. This yields the compact notation $V_\pi : \mathcal{S} \times \mathcal{O} \to \mathbb{R}$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π, the agent prefers being in state s with observation o over state s' with observation o'. Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}\Big[g(t)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}\Big[g(t)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \mathbb{E}\Big[r_1 + \gamma\, g(t_{2:})\Big] \\ &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big], \end{aligned}$$

If the expectation is expressed explicitly in terms of the distributions p and π, the Bellman equation for V_π is obtained:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid o) \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r \mid s, a) \Big[\, r + \gamma\, V_\pi(s', o') \,\Big]. \end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s, in order to compare different actions and determine which one is the most promising under a given policy π. This function is called the state-action value function $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. If $Q_\pi(s, a) > Q_\pi(s, a')$, then, under policy π, choosing action a in state s yields a higher expected return than choosing action a'. It is defined as:

$$Q_\pi(s, a) = \mathbb{E}\Big[\, r + \gamma\, g(t) \,\Big], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a).$$

The function Q_π can be related to V_π by recognizing that, after taking action a in state s, the remaining return from the next state s' is precisely $V_\pi(s')$. This yields:

$$\begin{aligned} Q_\pi(s,a) &= \mathbb{E}\Big[\, r + \gamma\, g(t) \,\Big], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a) \\ &= \mathbb{E}\Big[\, r + \gamma\, V_\pi(s', o) \,\Big] \\ &= \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r \mid s, a) \Big[\, r + \gamma\, V_\pi(s', o) \,\Big] \end{aligned}$$

1.2.5 Optimal Policy

The goal of reinforcement learning is to find a policy that maximizes, for every observation, the expected cumulative reward until a terminal state is reached. This policy is called the optimal policy and is denoted by π^*. Formally, π^* is a deterministic policy that always selects the action with the highest state-action value:

$$\pi^*(a \mid o) = \begin{cases} 1 & \text{if } a = \arg\max\limits_{a' \in \mathcal{A}} Q_{\pi^*}(s, a'), \\ 0 & \text{otherwise,} \end{cases}$$

where s is the true state underlying the observation o.

It is worth noting that defining the optimal policy in a partially observable setting is more delicate than in the fully observable case. Since the policy only has access to observations o rather than the true state s, the observation must carry enough information to distinguish between states that require different optimal actions. For the sake of simplicity, we assume throughout this manuscript that this condition holds. This assumption is of interest when one seeks to establish mathematical proofs, but in real-world implementations it matters little.

As just shown, both V_π and Q_π can be expressed explicitly in terms of the environment dynamics p and the policy π. If p and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o. This is possible precisely because V_π and Q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments of have too many states to allow an explicit representation of the dynamics p. Storing and computing over the full transition function becomes intractable. To overcome this limitation, instead of directly constructing the optimal policy, an iterative approach is adopted: starting from an initial policy, it is progressively improved so that it converges toward π^*. This iterative improvement through interaction with the environment is precisely what constitutes learning in reinforcement learning, and it is the subject of the next section.

1.3 Learning

Now that the optimal policy π^* has been defined, the practical question of how to obtain it arises. The recursive Bellman equations for V_π and Q_π suggest

dynamic programming approaches, but these scale poorly to large state spaces. Reinforcement learning offers an alternative: it seeks to solve the MDP iteratively, by generating a sequence of policies that progressively approach π^* through direct interaction with the environment. In this section, two major families of reinforcement learning methods used today are introduced: critic-only methods and actor-critic methods.

1.3.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π, denoted by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π, the policy always acts greedily with respect to this approximation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg\max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ 0 & \text{otherwise.} \end{cases}$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{Q}_\pi$.

Since $\hat{Q}_\pi$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{Q}_\pi : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$. Its objective is to approximate the true Q_π, which depends on the underlying states s, using only the partial information available through o. Once again, this poses no issue in practice, as it reflects how these algorithms are actually deployed in real-world applications.

To give a clear example of critic-only methods, they are presented in their simplest form: model-free, without temporal-difference learning, without replay buffer, without bootstrapping, and using only ε-greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a reinforcement learning algorithm, one of the first questions is how to represent the value function. In critic-only methods, this means choosing a model for $\hat{Q}_\pi$. Any machine learning model capable of regression task can be used for this purpose. The set of parameters of the model is denoted by θ. In deep reinforcement learning, a neural network is used to model this function, in which case θ represents the weights and biases of the network.

In our simple setting, exploration is handled via ε-greedy selection. With probability $1 - \varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. The trajectory distribution induced by the resulting policy now depends on both π and ε, and is denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted

over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^i and observation o_1, the interaction produces a episode t_e sampled from $\tau_t(s_1^i, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^{T} \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i).

2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input, and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. The improved approximation is denoted by $\hat{Q}'_\theta$, which now predicts returns more accurately under π. Since $\hat{Q}'_\theta$ has changed, the policy derived from it also changes. This new policy is denoted by π'.

Because the policy is now π', the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π'. This requires returning to step 1 to collect fresh interaction data generated by π'. The two steps thus alternate until the policy and the value approximation stabilize.

Even this simple algorithm already illustrates a difficulty inherent to all forms of reinforcement learning: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π'. For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict returns under π, remains reasonably accurate under π'. However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [**?**].

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [**?**]. In modern implementations, the value function is approximated by a neural network. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of reinforcement learning algorithms exists: actor-critic methods.

1.3.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, the objective is to approximate the state-value function V, instead of the state-action value function Q. Second, the policy π is given its own dedicated set of parameters. Two separate components are thus obtained: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2}.

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_{i:})$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2, it can be updated directly. The update uses the notion of advantage, defined as:

$$A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o),$$

where t is the trajectory that starts by taking action a in state s and then follows π for all subsequent steps.

When $A(s, a) > 0$, the action a performed better than expected, and the policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a \mid o)$. When $A(s, a) < 0$, the action a performed worse than expected, and the probability is decreased.

Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the critic provides a learned baseline for the actor, and the actor generates the data used to train the critic. This interplay reduces the instability inherent to reinforcement learning.

PPO (Proximal Policy Optimization), which is the current standard in reinforcement learning, belongs to this family of actor-critic methods. It will be used throughout this manuscript for all reinforcement learning experiments.

Now that highly performant policies can be trained, learning purely through interaction with their environment without any human supervision, the question arises of how to explain their behavior. This is the subject of the next chapter: explainability in the context of reinforcement learning.